

Programming SQL databases class 3

Definitions

Operators:

```
"=, !=, < <=, >, >="
```

Functions description:

- NULLIF() - returns NULL if two expressions are equal, otherwise it returns the first expression.
- CAST() - converts a value (of any type) into a specified datatype.

Query metadata

Extract table name with column names

```
SELECT TABLE_NAME, COLUMN_NAME  
FROM Table1.INFORMATION_SCHEMA.COLUMNS  
WHERE TABLE_NAME = 'Table1';
```

Query from single table

SELECT

Query all rows and columns from a table (select all columns)

```
SELECT * FROM t;  
SELECT * FROM Table1;
```

Query data in columns c1, c2 from a table

```
SELECT c1, c2 FROM t;  
-- Select columns "Index" and "Attribute1" from "Table1"  
SELECT Index, Attribute1 FROM Table1;
```

Query data and filter rows with a condition

```
SELECT c1, c2 FROM t WHERE condition;  
SELECT column1, column2 FROM table_name WHERE condition;
```

Selecting NULL or NOT NULL values

```
SELECT c1, c2 FROM t WHERE c1 IS NULL;  
-- reverse matching alternative  
SELECT c1, c2 FROM t WHERE c1 IS NOT NULL;
```

Query distinct rows from a table

```
SELECT DISTINCT c1 FROM t WHERE condition;  
-- example for rows greater than 10  
SELECT DISTINCT c1 FROM t WHERE c1 > 10;  
-- example for rows equal to 'active' but in column c2
```

Sort the result set in ascending or descending order

```
SELECT c1, c2 FROM t ORDER BY c1 ASC [DESC];
```

Skip offset of rows and return the next n rows

```
SELECT c1, c2 FROM t ORDER BY c1 LIMIT n OFFSET offset  
-- Select 2 rows after top 2 rows
```

```
SELECT * FROM Table1 LIMIT 2 OFFSET 2;
```

Group rows using an aggregate function

```
SELECT c1, aggregate(c2) FROM t GROUP BY c1;
```

Filter groups using HAVING clause

```
SELECT c1, aggregate(c2) FROM t GROUP BY c1 HAVING condition;
```

Select top 4 rows

```
SELECT TOP 4 * FROM Table1;
```

Select with alias

```
SELECT column_name AS alias_name FROM table_name;
```

Select not null values

```
SELECT column1, column2 FROM table_name WHERE column_name IS NOT NULL;
```

Query from many tables

Concatenate rows from two queries from two tables

A set operation that stacks rows vertically, assuming the queries produce compatible result sets. The columns selected (c1 and c2) must match in number, order, and data types across both queries—otherwise, the query fails with errors like "column type mismatch."

```
-- with duplicates
```

```
SELECT c1, c2 FROM t1 UNION ALL SELECT c1, c2 FROM t2;
```

```
-- without duplicates
```

```
SELECT c1, c2 FROM t1 UNION SELECT c1, c2 FROM t2;
```

Extract common rows from two queries from two tables

1. Compare rows entirely (all columns: c1 and c2 must match exactly).
2. Retain only rows present in both A and B.
3. Deduplicate: Use sorting, hashing, or temp tables to ensure uniqueness (e.g., if a row appears twice in one set, it's still only once in the result).
4. For INTERSECT ALL (if supported): Count occurrences and take the min per unique row.

```
SELECT c1, c2 FROM t1 INTERSECT SELECT c1, c2 FROM t2;
```

```
-- alternative
```

```
SELECT DISTINCT t1.c1, t1.c2
```

```
FROM t1
```

```
INNER JOIN t2
```

```
ON t1.c1 = t2.c1 AND t1.c2 = t2.c2;
```

JOIN

Aliases (t1 and t2) are used to qualify column names, preventing ambiguity if the same column name exists in both tables. Without aliases, you'd get an error in many SQL dialects if columns overlap.

Inner join t1 and t2

An INNER JOIN returns only the rows that have matching values in both tables according to the join condition. Non-matching rows from either table are excluded entirely from the result set. This makes it the most common join type for combining related data while filtering out

unpaired records. It's essentially the intersection of the two tables based on the predicate.

For each row in t1, search for rows in t2 where t1.column_name = t2.column_name. Only pairs that match are retained; non-matches are discarded.

Conceptually, it's like a filtered Cartesian product: generate all possible pairs (like CROSS JOIN), then keep only those satisfying the ON clause.

If ON is always true (e.g., ON 1=1), it becomes CROSS JOIN.

Includes all combinations (e.g., if two rows in t1 have same column_name matching three in t2, $2 \times 3 = 6$ rows).

Rows with NULL in join columns don't match (since NULL = NULL is false in SQL). Use IS NULL or COALESCE if needed.

```
SELECT c1, c2 FROM t1 INNER JOIN t2 ON condition;
-- Select 1 column from each table
SELECT t1.column1, t2.column2
FROM table1 t1
INNER JOIN table2 t2
ON t1.column_name = t2.column_name;
```

Left join t1 and t2 (Excel VLOOKUP plus rows duplication for one-to-many relationship)

A LEFT JOIN returns all rows from the left table (table1 here) and the matching rows from the right table (table2).

If one or more matches are found, each match creates a combined row in the result. Therefore if multiple rows in t2 match a single row in t1 (e.g., one-to-many relationship), the left row is duplicated in the result for each match.

If there are no matches for a row in the left table, the result still

includes that left row, but with NULL values in the columns from the right table.

This makes it useful for scenarios where you want to include all records from one side, even if the other side lacks corresponding data. It's an "outer" join because it preserves non-matching rows from one side (the left).

```
SELECT c1, c2 FROM t1 LEFT JOIN t2 ON condition;
-- Select 1 column from each table
SELECT t1.column1, t2.column2
FROM table1 t1
LEFT JOIN table2 t2
ON t1.column_name = t2.column_name;
```

Right join t1 and t2

All rows from the right table (t2) are included.

1. Read all rows from table2 (t2). No right rows are ever excluded.
2. For every row in t2, search for matching rows in t1 based on the ON condition (e.g., equal column_name).
 - If matches exist, create combined rows for each.
 - If no match, create a row with the t2 data and NULLs in t1 columns.
3. If multiple t1 rows match a t2 row (many-to-one), the right row is duplicated for each match.

```
SELECT c1, c2 FROM t1 RIGHT JOIN t2 ON condition;
-- Select 1 column from each table
SELECT t1.column1, t2.column2
FROM table1 t1
```

```
RIGHT JOIN table2 t2
ON t1.column_name = t2.column_name;
```

Full outer join

A FULL JOIN returns all rows from both tables, including matches and non-matches. For rows without a match in the other table, the missing columns are filled with NULL values.

1. Pair rows where ON condition is true (e.g., equal column_name).
2. For unmatched t1 rows: Include with NULLs in t2 columns.
3. For unmatched t2 rows: Include with NULLs in t1 columns.
 Duplicates if one-to-many (e.g., one t1 row matches multiple t2 → multiple rows).

This combines the behaviors of LEFT JOIN and RIGHT JOIN, providing a complete union of both datasets with matches where possible. Conceptually, it's the union of LEFT and RIGHT JOINS without duplicates.

```
SELECT c1, c2 FROM t1 FULL OUTER JOIN t2 ON condition;
-- Select 1 column from each table

SELECT t1.column1, t2.column2
FROM table1 t1
FULL JOIN table2 t2
ON t1.column_name = t2.column_name;
-- Equivalent with LEFT and RIGHT JOIN

SELECT t1.column1, t2.column2 FROM table1 t1 LEFT JOIN table2 t2 ON
t1.column_name = t2.column_name
UNION
SELECT t1.column1, t2.column2 FROM table1 t1 RIGHT JOIN table2 t2 ON
t1.column_name = t2.column_name;
```

Cross join a Cartesian product of rows in tables

For each row in table1, it pairs it with every single row in table2.

If Table A has m rows and Table B has n rows, the result will have $m \times n$ rows.

The SELECT clause then projects the specified columns from these paired rows.

CROSS JOIN does not require an ON condition. It combines every row from the first table with every row from the second table, regardless of any matching criteria.

```
SELECT c1, c2 FROM t1 CROSS JOIN t2;
```

```
-- Select 1 column from each table
```

```
SELECT t1.column1, t2.column2
```

```
FROM table1 t1
```

```
CROSS JOIN table2 t2;
```

```
-- Alternative old style syntax
```

```
SELECT c1, c2 FROM t1, t2
```

Intersection of data with JOIN. Finding overlaps for matching rows for columns c1 and c2 in both tables:

```
SELECT DISTINCT t1.c1, t1.c2
```

```
FROM t1
```

```
INNER JOIN t2
```

```
ON t1.c1 = t2.c1 AND t1.c2 = t2.c2;
```

```
-- alternative
```

```
SELECT c1, c2 FROM t1 INTERSECT SELECT c1, c2 FROM t2;
```

Selection combinations

A set subtraction between two result sets

It returns all unique rows from the first query (`SELECT c1, c2 FROM t1`) that are not present in the second query (`SELECT c1, c2 FROM t2`).

MINUS is used in Oracle, DB2, and some others. Identify rows in A that have no exact match in B (whole-row comparison: c1 and c2 must match). Deduplicate within the result. For EXCEPT ALL (if supported): Subtract counts (e.g., if row appears 3 times in A and 2 in B, result has 1).

Equivalent to EXCEPT in PostgreSQL, SQL Server, SQLite, MySQL (v8.0.31+), and ANSI SQL standard.

Edge Cases:

- No uniques in A: Empty.
- B empty: All unique from A.
- A subset of B: Empty.
- Duplicates in A: Removed if not in B.

```
SELECT c1, c2 FROM t1 MINUS SELECT c1, c2 FROM t2;
```

```
-- alternative
```

```
SELECT c1, c2 FROM t1 EXCEPT SELECT c1, c2 FROM t2;
```

```
-- alternative
```

```
SELECT t1.c1, t1.c2
```

```
FROM t1
```

```
LEFT JOIN t2 ON t1.c1 = t2.c1 AND t1.c2 = t2.c2
```

```
WHERE t2.c1 IS NULL;
```

Selection based on pattern matching in single column

The LIKE operator enables pattern matching on string data, allowing flexible searches for substrings or patterns rather than exact equality (e.g., via =). It uses two special wildcards:

- % (percent sign): Matches any sequence of zero or more characters (like a multi-character wildcard).
- _ (underscore): Matches exactly one character (like a single-character wildcard).

```

SELECT c1, c2 FROM t1 WHERE c1 LIKE pattern;
-- reverse matching alternative
SELECT c1, c2 FROM t1 WHERE c1 NOT LIKE pattern;
-- example for rows starting with 'abc'
SELECT c1, c2 FROM t1 WHERE c1 LIKE 'abc%';
-- example for rows having 'a_c' which matches "abc", "a1c" but not "ac" or "abcc"
SELECT c1, c2 FROM t1 WHERE c1 LIKE 'a_c';
-- example for rows having 'a_c' as substring matches (a, any character, b)
SELECT c1, c2 FROM t1 WHERE c1 LIKE '%a_b%';
-- example for rows having '100%' which requires escape character
SELECT c1, c2 FROM t1 WHERE c1 LIKE '100\%' ESCAPE '\';

```

Selection based on list

This performs a selection of rows from table t, retrieving columns c1 and c2, but only for rows where the value in c1 is (or is not, if NOT is included) present in the specified value_list.

The IN operator is a shorthand for checking membership in a set of values, equivalent to multiple OR conditions (e.g., c1 = val1 OR c1 = val2 ...).

NULL Issues:

- c1 NULL: Excluded.
- List has NULL: NOT IN always false if any NULL (can't confirm "not equal" to NULL).
- Fix: Filter NULLs (e.g., WHERE c1 NOT IN (...) AND c1 IS NOT NULL).

```

SELECT c1, c2 FROM t WHERE c1 IN value_list;
-- reverse matching alternative
SELECT c1, c2 FROM t WHERE c1 NOT IN value_list;
-- example for the list of 2 values
SELECT c1, c2 FROM t WHERE c1 IN ('apple', 'banana');

```

```
-- example for the list of 3 numerical values
```

```
SELECT c1, c2 FROM t WHERE c1 IN (1,2,3);
```

```
-- example with subquery
```

```
SELECT c1, c2 FROM t WHERE c1 IN (SELECT id FROM other_table WHERE status =
'active');
```

Production level query examples

Extracting storage locations (LOCNO) for material '12333209'

4 columns are returned. If no rows match, an empty result set:

- MATNR without leading zeros, aliased as Product (e.g., '000123' becomes '123').
- ML.MATID: Material ID from T_MATLOC.
- ML.LOCID: Location ID from T_MATLOC.
- L.LOCNO: Location number from T_LOC.

Declares three tables with aliases:

- ML: A material-location mapping table (e.g., where materials are stored).
- L: Location master table (e.g., warehouse or site details).
- MK: Material key/master table (e.g., material numbers and IDs).

WHERE with three parts (all must be true for a row to qualify):

- L.LOCID = ML.MATID: Joins locations to material-locations on ID (links T_LOC to T_MATLOC).
- MK.MATID = ML.MATID: Joins material keys to material-locations on material ID (links T_MATKEY to T_MATLOC).
- ML.MATID IN (subquery): Filters to only material IDs that match the subquery's results. Only rows where the material's trimmed MATNR is '12333209', and joined to locations.

```
SELECT
```

```
  REPLACE(LTRIM(REPLACE(MK.MATNR, '0', '')), '', '0') AS Product, -- Trim leading
```

zeros from MATNR

ML.MATID,

ML.LOCID,

L.LOCNO

FROM

[SAP].[T_MATLOC] AS ML

INNER JOIN

[SAP].[T_LOC] AS L ON L.LOCNO = ML.LOCID

INNER JOIN

[SAP].[T_MATKEY] AS MK ON MK.MATID = ML.MATID

WHERE

L.LOCID = ML.MATID

AND

MK.MATID = ML.MATID

AND

ML.MATID IN

(

SELECT MATID FROM

(

SELECT

REPLACE(LTRIM(REPLACE(MATNR,'0','')),','','0') Product,

MATID

FROM

[SAP].[T_MATKEY]

) A

WHERE PRODUCT IN ('12333209')

);

Author:

Zbigniew Galar